



iWAY

Introduction to JavaScript



IWAY Inc ©

Introduction

What is JS?

- JavaScript started as a scripting language for web browsers but has evolved into a full-fledged programming language capable of building complex applications.
- JavaScript was initially created to “make web pages alive”.
- JavaScript programs can be written right in a web page’s HTML and run automatically as the page loads.

Why is it called JavaScript?

- JavaScript was initially named “LiveScript”.
- Java was very popular at that time, so it was positioned as a “younger brother” of Java.

JS



JavaScript X Web Development

- JavaScript can execute in the browser, on the server, or on any device that has a JavaScript engine.
- Different engines have different “codenames”.
 - V8 – Chrome, Opera and Edge.
 - SpiderMonkey – Firefox.
 - Chakra – Internet Explorer,
 - “JavaScriptCore”, “Nitro” and “SquirrelFish” etc.
- What can in-browser JavaScript do?
 - Add new HTML to the page
 - Change the existing content, modify styles.
 - React to user actions
 - Run on mouse clicks, pointer movements, key presses.
 - Get and set cookies, ask questions to the visitor, show messages.
 - Remember the data on the client-side (“local storage”).



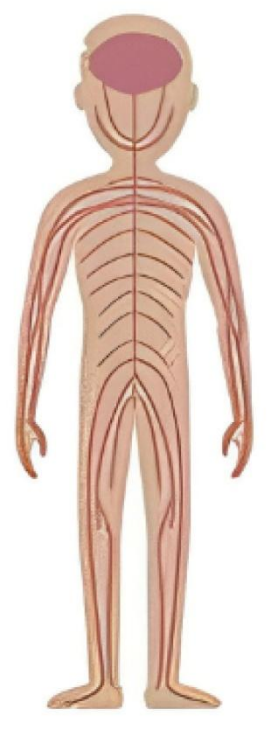
HTML the Skeleton



CSS the Skin



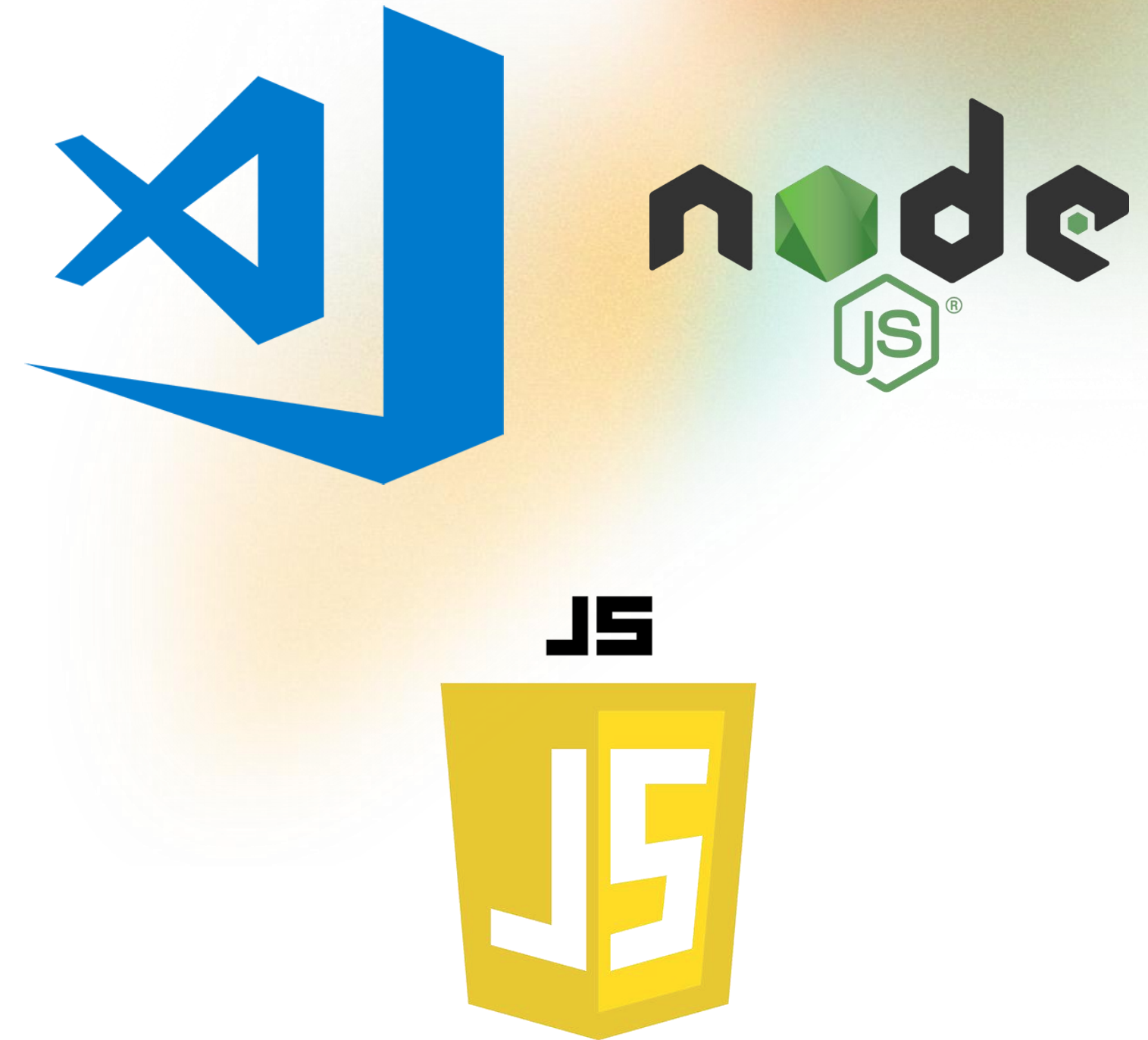
Javascript the Brain



Getting Started with JavaScript



- Install VS Code
- Install Node.js – <https://nodejs.org/>
- Open VS Code
- Open folder > Add file.js
- Write code
- Terminal > node file.js



Embed JavaScript in HTML

➤ Inline Script

- JavaScript code is written directly within `<script>` tags in an HTML file.

```
index.html
<script>
    alert('Hello, world!');
</script>
```

➤ External Script

- JavaScript code is stored in an external file and linked using the `src` attribute.
- This improves code organization and reusability.

```
index.html
<script src="HelloWorld.js"></script>
```

Code Structure

➤ Line Breaks & Readability

- JavaScript is case-sensitive and follows a structured approach.
- Use proper indentation and spacing for clarity.

➤ Comments in JavaScript

- Single-line comments
- Multi-line comments

➤ Semicolons (;) in JavaScript

- Semicolons are optional in JavaScript.
- However, it is recommended to explicitly use semicolons to avoid unexpected errors.

```
hello.js

let x = 10;
let y = 20;
console.log(x + y);
```

```
hello.js

// This is a single-line comment

/*
    This is line 1 of multi-line comment
    This is line 2 of multi-line comment
*/
```


JavaScript Data Types

➤ JavaScript has two main types:

- Primitive
- Object (Non-primitive)

➤ Primitive Data Types

Type	Example
Number	let x = 10;
String	let name = "John";
Boolean	let isActive = true;
Null	let emptyValue = null;
Undefined	let notDefined;

➤ Objects (Non-primitive)

```
hello.js  
  
let person = { name: "John", age: 30 };
```

typeof Operator

Used to determine the data type of a variable.

```
hello.js

console.log(typeof 42);           // "number"
console.log(typeof "hello");      // "string"
console.log(typeof true);         // "boolean"
console.log(typeof null);         // "object" (special case)
console.log(typeof undefined);    // "undefined"
console.log(typeof {});           // "object"
console.log(typeof []);           // "object"
```


String Conversion

Any value can be converted to a string using String().

```
hello.js

let value = true;
console.log(typeof value); // boolean
value = String(value); // now value is a string "true"
console.log(typeof value); // string
```

Numeric Conversion

Implicit Numeric Conversion (Type Coercion)

- JavaScript automatically converts strings to numbers in mathematical operations.

```
hello.js  
  
console.log("6" / "2"); // 3, strings are converted to numbers
```

Explicit Numeric Conversion

- Use `Number()` to convert a string to a number.

```
let str = "123";  
console.log(typeof str); // string  
let num = Number(str);  
console.log(typeof num); // number
```

If the string is not a valid number, it converts to NaN.

Boolean Conversion

Some values are falsy (convert to false):

- 0, "" (empty string), null, undefined, NaN
- All other values are truthy (convert to true).

```
hello.js

console.log(Boolean(1)); // true
console.log(Boolean(0)); // false
console.log(Boolean("hello")); // true
console.log(Boolean("")); // false
```

Basic Operators – Math



Operators Overview

- Operands: The values operators act upon.
- Unary Operator: Operates on one value.
- Binary Operator: Operates on two values.

Arithmetic Operators

Operator	Description	Example
+	Addition	$5 + 2 = 7$
-	Subtraction	$5 - 2 = 3$
*	Multiplication	$5 * 2 = 10$
/	Division	$5 / 2 = 2.5$
%	Remainder	$5 \% 2 = 1$
**	Exponentiation	$2 ** 3 = 8$

String Concatenation with +

When adding a string and a number, JavaScript converts the number to a string.

```
hello.js  
  
console.log("Hello" + " World"); // "Hello World"  
console.log("1" + 2); // "12"  
console.log(2 + "1"); // "21"
```

To ensure numeric addition, use Number() or unary +

```
hello.js  
  
console.log(+ "5" + 2); // 7
```

Assignment Operator =

The = operator is used to assign values to variables

```
● ● ●  
  
//Single Assignment  
let x = 10;  
console.log(5);  
  
//Chaining assignments  
let a, b, c;  
a = b = c = 2 + 2;  
console.log(a, b, c); // 4 4 4
```


Modify-in-Place Operators

Shorthand for modifying values

```
let num = 10;  
num += 5; // Equivalent to num = num + 5;  
num -= 3; // Equivalent to num = num - 3;  
num *= 2; // Equivalent to num = num * 2;  
num /= 3; // Equivalent to num = num / 3;  
num %= 2; // Equivalent to num = num % 2;
```

Increment & Decrement Operators



- Used to increase or decrease a number.
- Can be prefix (++x) or postfix (x++).

Operator	Description
++x	Increments before returning the value
x++	Increments after returning the value
--x	Decrements before returning the value
x--	Decrements after returning the value

```
let a = 1;
console.log(++a); // 2 (prefix: increments first)
console.log(a++); // 2 (postfix: returns first, then increments)
console.log(a);   // 3
```


Comparisons in JavaScript

JavaScript provides several comparison operators:

Operator	Description	Example	Result
>	Greater than	5 > 3	TRUE
<	Less than	5 < 3	FALSE
>=	Greater than or equal to	5 >= 5	TRUE
<=	Less than or equal to	5 <= 4	FALSE
==	Equal to (loose comparison)	'5' == 5	TRUE
===	Equal to (strict comparison)	'5' === 5	FALSE
!=	Not equal to (loose comparison)	5 != 3	TRUE

String comparisons

- Strings are compared character by character based on their Unicode values.
- Rules for comparison:
 - Compare the first character of both strings.
 - If they are equal, move to the next character.
 - Repeat until a difference is found or one string ends.
 - If one string is longer, it is considered greater.

```
hello.js  
  
console.log("apple" > "apricot"); // false ('p' < 'r')  
console.log("hello" < "Hello"); // false ('h' > 'H')
```


Comparison of Different Types

➤ When comparing different data types, JavaScript converts them into numbers.

⚠ *Be careful with implicit conversions!*

✅ *Use === instead of == to avoid unwanted conversions.*



hello.js

```
console.log('5' > 2); // true ('5' is converted to number)
console.log(true == 1); // true (true is converted to 1)
console.log(false == 0); // true (false is converted to 0)
```

Strict Equality (===)

- Regular equality (==) allows type conversion, which can lead to unexpected results.
- Strict equality (===) does NOT perform type conversion.

```
hello.js

console.log(0 == false); // true (type conversion happens)
console.log(0 === false); // false (different types)
console.log('5' == 5); // true (string converted to number)
console.log('5' === 5); // false (different types)
```


Conditional Branching – if, else

- if statement: Executes code if the condition is true.

```
hello.js

let age = 18;
if (age >= 18) {
    console.log("You are an adult.");
}
```

- else clause: Runs if the condition is false.

```
hello.js

if (age >= 18) {
    console.log("You are an adult.");
} else {
    console.log("You are a minor.");
}
```

Multiple conditions: else if

- Else – if ladder is used to evaluate multiple conditions

```
hello.js

if (age < 12) {
    console.log("Kid");
} else if (age < 18) {
    console.log("Teen");
} else {
    console.log("Adult");
}
```


Conditional Operator (?)

- The ternary operator (?) is a shorthand for if-else.
- The ternary operator (?) is a shorthand for if-else.



syntax.js

```
let result = condition ? value1 : value2;
```



example.js

```
//Single condition
```

```
let person = (age < 18) ? 'Minor' : 'Adult';
```

```
//Multiple conditions
```

```
let person = (age < 12) ? 'Kid' :  
              (age < 18) ? 'Teen' :  
              (age < 60) ? 'Adult' : 'Senior';
```

The switch statement

- The switch statement is an alternative to multiple if..else conditions.

```
hello.js

switch (expression) {
  case value1:
    // Code to execute if expression === value1
    break;
  case value2:
    // Code to execute if expression === value2
    break;
  default:
    // Code to execute if no match is found
}
```

```
let a = 2 + 2;
switch (a) {
  case 3:
    console.log('Too small');
    break;
  case 4:
    console.log('Exactly!');
    break;
  case 5:
    console.log('Too big');
    break;
  default:
    console.log("I don't know such values");
}
```


Logical Operator – OR (||)

- Returns true if one or more inputs are true

```
hello.js  
  
console.log(4 > 7 || 6 == "6"); // true  
console.log(4 > 7 || 6 == 7); // false
```

- Returns the first truthy value or the last value if all are falsy.

```
console.log(0 || null || "Hello" || undefined); // "Hello"  
console.log(0 || null || "" || undefined); // undefined
```

Logical Operator – AND (&&)

- Returns true if and only if all inputs are true

```
console.log(10 > 5 && 8 < 12); // true  
console.log(10 > 5 && 8 == 12); // false
```

- Returns the first falsy value or the last value if all are truthy.

```
console.log(5 && "Hi" && 0 && true); // 0  
console.log(5 && "Hi" && 1 && true); // true
```


Logical Operator – Not (!)

- Converts the value to boolean and inverts it.

```
console.log(!true); // false  
console.log(!0); // true
```

Loops in JavaScript

- A loop is a programming construct that repeats a block of code while a condition is true.
- Loops help automate repetitive tasks, reducing code duplication.
- Types of Loops in JavaScript:
 - while Loop
 - do...while Loop
 - for Loop

While Loop

- Runs while the condition is true.

```
let i = 3;
while (i > 0) {
  console.log(i);
  i--;
}
```

Do...while Loop

- Runs at least once, then checks the condition.

```
let num = 0;  
do {  
  console.log(num);  
  num++;  
} while (num < 3);
```


For Loop

- Iterator declaration, condition and increment / decrement are a part of the loop's syntax.

```
for (let i = 0; i <= 3; i++) {  
  console.log(i);  
}
```

Loop Control Statements

➤ break → Exits the loop immediately.

```
for (let i = 1; i <= 10; i++) {  
  if (i % 3 === 0) break;  
  console.log(i);  
}  
// Output: 1, 2
```

➤ continue → Skips the current iteration.

```
for (let i = 1; i <= 10; i++) {  
  if (i === 3) continue;  
  console.log(i);  
}  
// Output: 1, 2, 4, 5, 7, 8, 10
```


Functions in JavaScript

- Functions are reusable blocks of code that perform a specific task.
- Functions help make code modular and easier to manage.
- Function declaration:

```
function greet() {  
    console.log(`Welcome, user!`);  
}  
greet();
```

Variables in functions

- Local Variables: Variables inside a function are not accessible outside.

```
function showMessage() {  
  let message = "Hello";  
  console.log(message);  
}  
console.log(message); // Error
```

- Outer Variables: Functions can access variables from the outer scope.

```
let user = "Alice";  
function sayHello() {  
  console.log(`Hello, ${user}!`);  
}  
sayHello();
```


Function Parameters

- Parameters: We can pass arbitrary data to functions using parameters.

```
function greet(name) {  
    console.log(`Hello, ${name}!`);  
}  
greet('Ravi'); // Hello, Ravi!
```

- Default parameter:

- If a function is called without an argument, the parameter defaults to undefined.
- We can set a default value using = in the function declaration.

```
function greet(name = "Guest") {  
    console.log(`Hello, ${name}!`);  
}  
greet(); // Hello, Guest!
```

Returning values

➤ Returning a value:

- A function can return a value to the caller.
- Using return without a value exits the function immediately.

```
function add(a, b) {  
    return a + b;  
}  
console.log(add(5, 3)); // 8
```

- Functions represent actions, so their names should usually be verbs.
- They should be brief, accurate, and descriptive for clarity.
- Examples:
 - ❌ Bad: doStuff(), stuffHandler(), x()
 - ✅ Good: calculateTotal(), getUserData(), sendEmail()

Arrow functions in JavaScript

- Arrow functions provide a shorter and cleaner way to write functions in JavaScript.

```
let func = (arg1, arg2, ..., argN) => expression;
```

- Key Differences from Regular Functions:
 - No function keyword.
 - Shorter syntax for simple operations.

Single-Line Arrow Functions

- Single parameter (No parenthesis required)

```
let square = x => x * x;  
console.log(square(5)); // 25
```

- Multiple Parameters (Parentheses required)

```
let add = (a, b) => a + b;  
console.log(add(3, 7)); // 10
```

- No Arguments (Parentheses required)

```
let greet = () => console.log("Hello, world!");  
greet(); // "Hello, world!"
```


Multiline Arrow Functions

- When an arrow function has multiple lines:
 - `{}` should be used
 - Value should be returned explicitly

```
let multiply = (a, b) => {  
    let result = a * b;  
    return result;  
};  
console.log(multiply(4, 3)); // 12
```

Introduction to Objects

➤ Primitive vs Objects:

- Primitive data types hold single values (e.g., numbers, strings, booleans).
- Objects store keyed collections of various data and more complex entities.

object Declaration

```
let user = new Object(); // Constructor syntax
let user = {}; // Object literal syntax
```

object Example

```
let user = {
  name: "John", // Key "name", Value "John"
  age: 30       // Key "age", Value 30
};
```


Object Properties

➤ Key-Value Pairs:

- Key: Property name (must be a string or symbol).
- Value: Can be of any type.

➤ Accessing Properties:

```
console.log(user.name); // Dot notation  
console.log(user["age"]); // Bracket notation
```

➤ Removing Properties:

```
delete user.age;
```

Dynamic Keys in Objects

➤ Using Variables as Keys

```
let key = "profession";
let person = {
  name: "Alice",
  [key]: "Engineer" // Computed property name
};
console.log(person.profession); // Engineer
```

➤ Property Existence Test (in Operator)

```
console.log("name" in user); // true
console.log("salary" in user); // false
```


Looping Through Objects

- The for..in Loop is used to iterate over all keys in an object.

```
program.s

let user = {
  name: "John",
  age: 30,
  isAdmin: true
};

for (let key in user) {
  console.log(`${key}: ${user[key]}`);
}

/*
output:
name: John
age: 30
isAdmin: true
*/
```

Strings in JavaScript

- Strings are used for storing text.
- They can be enclosed in Single Quotes, Double Quotes and Backticks
- Escape sequences:
 - `\n` → Newline
 - `\t` → Tab
 - `\'` → Single quote
 - `\"` → Double quote
 - `\`` → Backtick
 - `\\` → Backslash

Strings functions - 1 / 5

➤ String Length

```
program.js  
  
let str = "Hello";  
console.log(str.length); // 5
```

➤ Accessing characters

```
program.js  
  
console.log(str[0]); // 'H'  
console.log(str.charAt(1)); // 'e'
```

Strings functions - 2 / 5

➤ Convert case

program.js

```
console.log("hello".toUpperCase()); // "HELLO"  
console.log("WORLD".toLowerCase()); // "world"
```


Strings functions - 3 / 5

➤ indexOf(substr, pos)

- Finds the first occurrence of 'substr' in the string, starting from 'pos' (optional).
- Returns the index if found, otherwise -1.

```
program.js

let str = "Hello, world!";
console.log(str.indexOf("o"));           // Output: 4
console.log(str.indexOf("o", 5));        // Output: 8 (search starts from index 5)
console.log(str.indexOf("xyz"));         // Output: -1 (not found)
```

➤ lastIndexOf(substr, position)

- Finds the last occurrence of 'substr', searching backward from 'position' (optional)
- Returns the index if found, otherwise -1.

```
program.js

console.log(str.lastIndexOf("o"));       // Output: 8 (last occurrence of 'o')
console.log(str.lastIndexOf("o", 7));    // Output: 4 (searches up to index 7)
console.log(str.lastIndexOf("xyz"));     // Output: -1 (not found)
```

Strings functions - 4 / 5

- includes(substr, pos)
 - Checks if 'substr' exists in the string, starting from 'pos' (optional).
 - Returns true if found, otherwise false.

```
program.js

let str2 = "JavaScript is awesome!";
console.log(str2.includes("Script")); // Output: true
console.log(str2.includes("script")); // Output: false (case-sensitive)
console.log(str2.includes("is", 10)); // Output: false (search starts from index 10)
```

- startsWith(substr, pos)
 - Checks if the string starts with 'substr', optionally from 'pos'.
 - Returns true if it matches, otherwise false.

```
program.js

console.log(str2.startsWith("Java")); // Output: true
console.log(str2.startsWith("Script")); // Output: false
console.log(str2.startsWith("Script", 4)); // Output: true (search starts at index 4)
```


Strings functions - 5 / 5

➤ endsWith(substr, length)

- Checks if the string ends with 'substr', considering only the first 'length' characters (optional)
- Returns true if it matches, otherwise false.

```
let str2 = "JavaScript is awesome!";  
console.log(str2.endsWith("awesome!")); // Output: true  
console.log(str2.endsWith("is", 13));   // Output: true (checks first 13 chars)  
console.log(str2.endsWith("Java"));     // Output: false
```

➤ substring(startindex, endindex)

- extracts part of a string.

```
console.log("Hello".substr(1, 3)); // "ell"
```

Arrays in JavaScript

- Arrays store collections of elements.
- Arrays can hold multiple data types, including numbers, strings, and objects.
- Arrays are indexed, starting from 0.
- Arrays are dynamic in JavaScript, meaning they can grow or shrink.
- Arrays in JavaScript do not have contiguous memory allocation.
- Array declaration in JavaScript:

```
let arr = new Array();  
let arr = [];  
let arr = [1, 2, 3];
```

- Array length:

```
let arr = [1, 2, 3];  
console.log(arr.length); // Output: 3
```


Accessing array elements

- Using indices (arr[i]):

```
let arr = ["apple", "banana", "cherry"];  
console.log(arr[0]); // Output: apple
```

- .at(i) supports negative indexing:

```
console.log(arr.at(-1)); // Output: cherry  
console.log(arr.at(-2)); // Output: banana
```

Adding / Removing Elements

- `push(value)`: Adds an element at the end.
- `pop()`: Removes the last element and returns it.
- `unshift(value)`: Adds an element at the beginning.
- `shift()`: Removes the first element and returns it.

```
let arr = [1, 2, 3];  
arr.push(4); // [1, 2, 3, 4]  
arr.pop(); // [1, 2, 3]  
arr.unshift(0); // [0, 1, 2, 3]  
arr.shift(); // [1, 2, 3]
```


Looping Over an Array

- Any of the basic loops can be used: while, do...while, for
- For loop can be used in two distinct ways when iterating over an array.
-  for...in is not recommended for arrays.

```
let arr = ["a", "b", "c"];

//1st implementation
for (let i = 0; i < arr.length; i++) {
  console.log(arr[i]);
}

//2nd implementation
for (let item of arr) {
  console.log(item);
}
```

foreach loop

➤ forEach(func)

- Calls a function for each array element.

```
let arr = ["apple", "banana", "cherry"];  
arr.forEach(item => console.log(item));
```

```
/*
```

```
Output:
```

```
    apple
```

```
    banana
```

```
    cherry
```

```
*/
```


Array methods - add / remove items

- splice(pos, deleteCount, ...items)
 - at index pos deletes deleteCount elements and inserts items.
- slice(start, end)
 - creates a new array, copies elements from index start till end (not inclusive) into it.
- concat(...items)
 - Merges arrays and returns a new array.

```
let arr = ["apple", "banana", "cherry"];  
arr.splice(1, 1, "new"); // [ 'apple', 'new', 'cherry' ]  
let subArr = arr.slice(1, 3); // Extracts elements from index 1 to 2  
let newArr = arr.concat("guava", "orange"); //[ 'apple', 'new', 'cherry', 'guava', 'orange' ]
```

Array methods - search in Array

- indexOf/lastIndexOf(item, pos)
 - looks for item starting from position pos, and return the index or -1 if not found.
- includes(value)
 - returns true if the array has value, otherwise false.
- find/filter(func)
 - filter elements through the function, return first/all values that make it return true.

```
let arr = [1, 2, 3, 2];  
console.log(arr.indexOf(2)); // Output: 1  
console.log(arr.lastIndexOf(2)); // Output: 3  
console.log(arr.includes(3)); // Output: true  
let result = arr.find(x => x > 2); // Output: 3
```


Array methods - Transform Array

➤ map(func)

- creates a new array from results of calling func for every element.

```
let arr = [1, 2, 3];  
let doubled = arr.map(x => x * 2);  
console.log(doubled); // [2, 4, 6]
```

➤ reverse()

- reverses the array in-place, then returns it.

```
let arr = [1, 2, 3];  
arr.reverse();  
console.log(arr); // [3, 2, 1]
```

Array methods - Sort

- Default Behavior (array.sort())
 - If no compare function is provided, the array elements are converted to strings and sorted in lexicographic (dictionary) order.

```
let numbers = [25, 100, 2, 50, 10];
numbers.sort();
console.log(numbers);
// Output: [ 10, 100, 2, 25, 50 ] ❌ (Incorrect for numbers)
```

- Sorting with a Compare Function (array.sort(func))
 - To sort numbers correctly, we must pass a compare function.

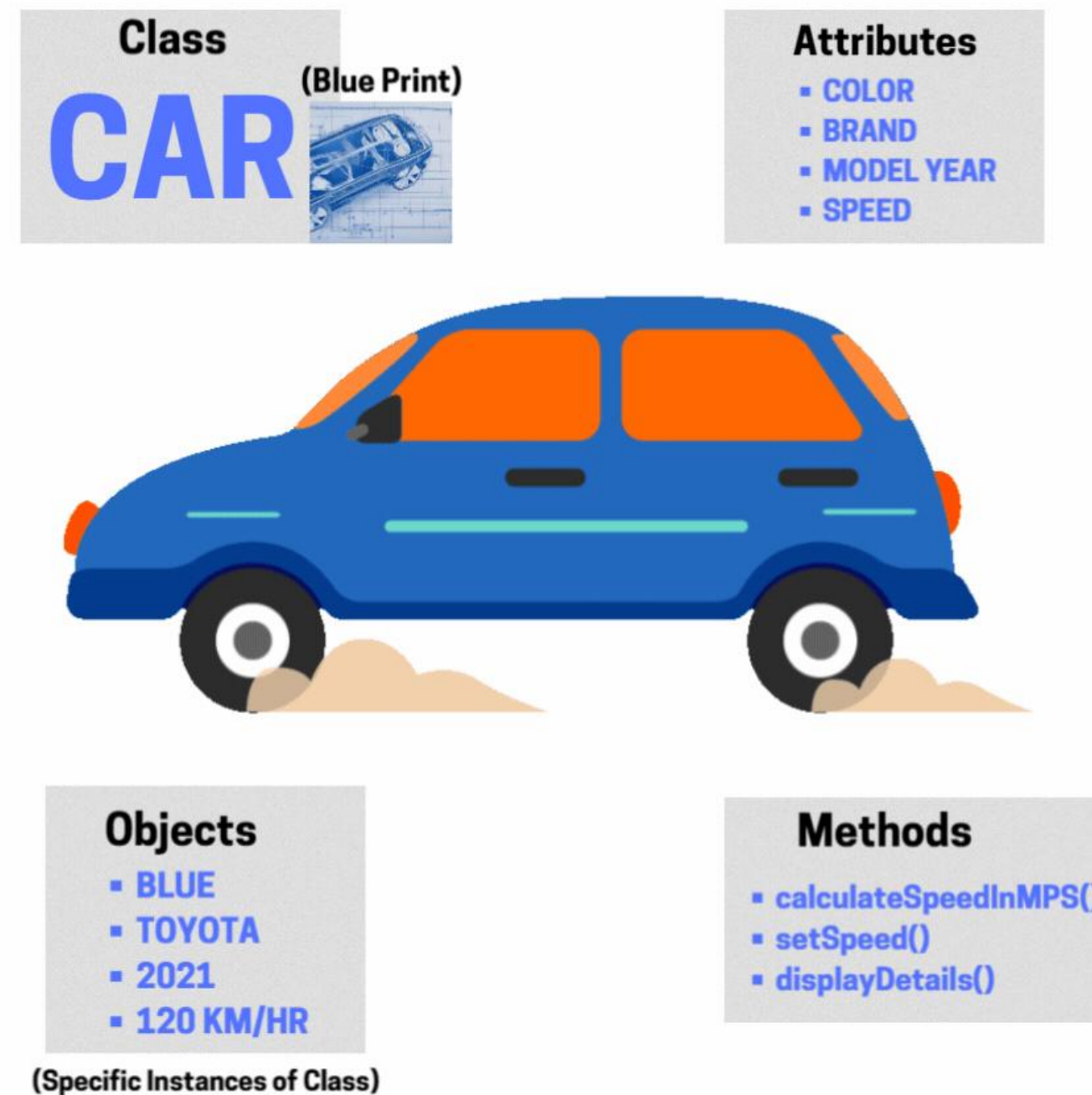
```
let numbers = [25, 100, 2, 50, 10];

// Ascending order
numbers.sort((a, b) => a - b);
console.log(numbers); // Output: [2, 10, 25, 50, 100]

// Descending order
numbers.sort((a, b) => b - a);
console.log(numbers); // Output: [100, 50, 25, 10, 2]
```


Classes in JavaScript

- JavaScript classes provide a blueprint for creating objects.
- Introduced in ES6, they offer a more structured way to implement object-oriented programming.
- Classes encapsulate properties and methods that define an object's behavior.



Class syntax

➤ Class Syntax:

```
class User {  
  greet() {  
    console.log("Hello, User!");  
  }  
}  
  
let obj = new User();  
obj.greet(); // Output: Hello, User!
```


Constructors in JavaScript

- The constructor method is called when a new object is created.
- It initializes object properties.

```
class User {  
  name;  
  constructor(name) {  
    this.name = name;  
  }  
  greet() {  
    console.log(`Hello, ${this.name}!`);  
  }  
}  
  
let obj = new User("John");  
obj.greet(); // Output: Hello, John!
```

Access Modifiers in JavaScript



- **Public:** Accessible from anywhere.
- **Private (#):** Accessible only inside the class.
- **Protected** (Not natively supported in JS but can be simulated using conventions like `_property`).

```
class User {  
  name;  
  #password;  
  constructor(name, password) {  
    this.name = name;  
    this.#password = password;  
  }  
  checkPassword(input) {  
    return input === this.#password;  
  }  
}  
  
let user = new User("Alice", "1234");  
console.log(user.checkPassword("1234")); // true  
console.log(user.#password); // Error: Private field
```


Class Inheritance

- The extends keyword is used to inherit from another class.
- The child class gets access to all properties and methods of the parent class.

```
class User {
  name;
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}!`);
  }
}

class Admin extends User {
  isAdmin = true;
}

let admin = new Admin("John");
admin.greet(); // Output: Hello, John!
console.log(admin.isAdmin);
```

Function Overriding

- If a child class defines a method with the same name as a parent class method, it overrides the parent method.
- The super keyword can be used to call the parent method inside the child class method.

```
class User {  
  greet() {  
    console.log("Hello, User!");  
  }  
}  
  
class Admin extends User {  
  greet() {  
    super.greet(); // Calls the parent class method  
    console.log("Hello, Admin!");  
  }  
}  
  
let admin = new Admin();  
admin.greet();  
// Output:  
// Hello, User!  
// Hello, Admin!
```


Constructor Overriding

- If a child class defines its own constructor, it overrides the parent's constructor.
- `super` is used to call the parent class constructor.

```
class User {
  name;
  constructor(name) {
    this.name = name;
  }
  greet() {
    console.log(`Hello, ${this.name}!`);
  }
}

class Admin extends User {
  role;
  constructor(name, role) {
    super(name); // Calls parent constructor
    this.role = role;
  }
  greet() {
    console.log(`Welcome, Admin ${this.name}! Your role is ${this.role}.`);
  }
}

let admin = new Admin("John", "Superuser");
admin.greet();
// Output: Welcome, Admin John! Your role is Superuser.
```

Classes – summary

- Classes simplify object creation and organization.
- constructor() initializes class properties.
- Access modifiers control property visibility (public, private).
- extends enables inheritance, and super calls parent methods.
- Constructor and function overriding allow customization in child classes.

JSON – JavaScript Object Notation



- JSON (JavaScript Object Notation) is a lightweight data-interchange format.
- It is easy to read, write, and parse for both humans and machines.
- Used to store and exchange data between a server and a client.
- **JSON Syntax**
 - JSON is written in key-value pairs.
 - Keys must be strings enclosed in double quotes.
 - Values can be strings, numbers, booleans, arrays, or other objects.

```
{  
  "name": "John",  
  "age": 30,  
  "isStudent": false,  
  "skills": ["JavaScript", "React", "Node.js"]  
}
```

Converting Object to JSON

- JSON.stringify() is used to convert a JavaScript object into a JSON string.
- Useful for sending data to a server.

```
let user = { name: "Bob", age: 28 };  
let jsonString = JSON.stringify(user);  
console.log(jsonString); // Output: '{"name":"Bob","age":28}'
```


Converting JSON to JavaScript Object



- JSON.parse() is used to convert a JSON string into a JavaScript object.
- Useful when receiving data from a web server.
- JSON supports automatic type conversion.

```
let jsonString = '{"name":"Alice","age":25}';
let user = JSON.parse(jsonString);
console.log(`user's name = ${user.name} | type of username = ${typeof user.name}`);
console.log(`user's age = ${user.age} | type of age = ${typeof user.age}`);
```

```
/* Output:
user's name = Alice | type of username = string
user's age = 25 | type of age = number
*/
```

Document Object Model



- The Document Object Model (DOM) represents the structure of an HTML document.

```
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Portfolio</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <header id="header">
    <h1 class="site-title">Akashdeep Srivastava</h1>
    <nav class="nav-links">
      <a href="#about" class="nav-item">About</a>
      <a href="#projects" class="nav-item">Projects</a>
      <a href="#contact" class="nav-item">Contact</a>
    </nav>
  </header>

  <section id="practice">
    <h2 class="section-title" id="about-title">Practice Section</h2>
    <p class="about-text">This section on this page is dedicated to JavaScript practice.</p>
    <input type="text" id="enterText" class="form-input" placeholder="Enter your text here!"
required />
    <button id="clickHere-btn">Click Here!</button>
    <script src="keyboardEvents.js"></script>
    <script src="mouseEvents.js"></script>
  </section>
</body>
</html>
```

```
document
├─ html
│   ├─ head
│   │   ├─ meta (charset="UTF-8")
│   │   ├─ meta (viewport)
│   │   ├─ title ("Portfolio")
│   │   └─ link (rel="stylesheet", href="styles.css")
│   └─ body
│       ├─ header#header
│       │   ├─ h1.site-title ("Akashdeep Srivastava")
│       │   └─ nav.nav-links
│       │       ├─ a.nav-item (href="#about", "About")
│       │       ├─ a.nav-item (href="#projects", "Projects")
│       │       └─ a.nav-item (href="#contact", "Contact")
│       └─ section#practice
│           ├─ h2.section-title#about-title ("Practice Section")
│           ├─ p.about-text ("This section on this page is dedicated ...")
│           ├─ input#enterText.form-input (type="text", placeholder="...")
│           └─ button#clickHere-btn ("Click Here!")
```


Document Object Model X JS

- JavaScript can interact with and modify the DOM dynamically.
- The Browser Object Model (BOM) provides additional objects to interact with the browser:
 - Screen: Provides screen dimensions.
 - Navigator: Gives browser details.
 - Location: Provides current URL details.
 - History: Contains the session history.

```
console.log(`Screen Height: ${screen.height}`);  
console.log(`Screen Width: ${screen.width}`);  
console.log(`Browser Information: ${navigator.userAgentData}`);  
console.log(`History Length: ${history.length}`);  
console.log(`Host Name: ${location.hostname}`);  
console.log(`Current URL: ${location.href}`);
```

Understanding the DOM Structure

- The document is the root node.
- The first child of document is <html>.
- The first child of <html> is <head>.

```
console.log(document); // Entire Document
console.log(document.firstChild); // <html>
console.log(document.firstChild.firstChild); // <head>
```


Querying and Selecting Elements

➤ Methods to select elements:

- `getElementsByTagName(x)`: Selects elements by tag name.
- `getElementsByClassName(x)`: Selects elements by class name.
- `getElementById(x)`: Selects an element by its ID.

```
let paragraphs = document.getElementsByTagName('p');  
let buttons = document.getElementsByClassName('btn');  
let mainHeader = document.getElementById('header');
```

Traversing the DOM

- Navigate through parent, child, and sibling elements.
 - Parent Node: `x.parentNode`
 - Child Nodes: `x.firstChild`, `x.lastChild`, `x.childNodes`
 - Sibling Nodes: `x.previousSibling`, `x.nextSibling`

```
let element = document.getElementById('example');  
console.log(element.parentNode); // Parent element  
console.log(element.firstChild); // First child node  
console.log(element.nextSibling); // Next sibling node
```


Creating an Element in the DOM

- Use `document.createElement(tagName)` to create a new element.
- Append the element to the document.

```
let newElement = document.createElement('p');  
newElement.textContent = 'This is a new paragraph';  
document.body.appendChild(newElement);
```

Updating an Element in the DOM

- Modify an element's content using JavaScript.

```
let contactNode = document.getElementById('contact');  
contactNode.textContent = 'Contact IWAY';
```


Removing an Element in the DOM

- Use `removeChild()` to delete an element.

```
let parent = document.getElementById('parentNode');  
let child = document.getElementById('childNode');  
parent.removeChild(child);
```

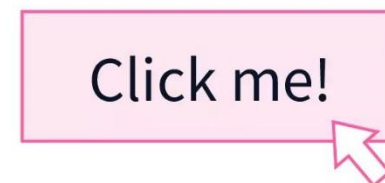
JavaScript Events

➤ Events are actions or occurrences in the browser that scripts can respond to.

➤ Common examples:

- Clicking a button
- Hovering over an element
- Pressing a key

1. User interacts with the page



2. An “Event” occur



3. A piece of JS code runs in reponse

```
function myEvent() {  
  ...  
}
```

4. The page's appearance is updated/modified in some way as a result

Types of Events – Mouse Events

- click – When an element is clicked.
- dblclick – When an element is double-clicked.
- mouseover – When the mouse enters an element.
- mouseout – When the mouse leaves an element.
- mousedown – When a mouse button is pressed down.
- mouseup – When a mouse button is released.

```
let button = document.getElementById('btn1');
button.addEventListener("click", buttonClick);
function buttonClick(){
    document.write("You have clicked the button!");
}
```

Types of Events – Keyboard Events

- keydown – When a key is pressed down.
- keyup – When a key is released.
- keypress (deprecated) – When a key is pressed (use keydown instead).

```
let inputField = document.getElementById('input1');
inputField.addEventListener("keydown", keyPress);
function keyPress(event){
    console.log("Key pressed: " + event.key);
}
```


Types of Events – Form Events

- submit – When a form is submitted.
- change – When an input field value changes.
- focus – When an input field is focused.
- blur – When an input field loses focus.

```
let form = document.getElementById('myForm');
form.addEventListener("submit", formSubmit);
function formSubmit(event){
    event.preventDefault();
    document.write("Form Submitted!");
}
```

Types of Events – Window Events

- load – When the page is fully loaded.
- resize – When the browser window is resized.
- scroll – When the page is scrolled.

```
window.addEventListener("resize", windowResize);  
function windowResize(){  
    console.log("Window resized to: " + window.innerWidth + "x" + window.innerHeight);  
}
```




iWAY

Thank you!



IWAY Inc ©